Desenvolvimento do Plano de Verificação Versão 1.3

Mini GPU SIMD Simplificada

Vinícius Carvalho Pimpim

Projeto Fase 2 - CI Digital

1 Especificação

O IP a ser desenvolvido no projeto final da Fase 2 consiste em uma GPU simplificada baseada na lógica SIMD (*Single Instruction, Multiple Data*). Nessa técnica de computação paralela, uma mesma operação é aplicada simultaneamente a diferentes elementos de dados. O objetivo do projeto é desenvolver um módulo capaz de acelerar cálculos vetoriais simples por meio de quatro unidades aritméticas operando em paralelo.

A GPU recebe dois vetores de quatro posições, com elementos de entrada *signed* de 8 bits em complemento de dois. Para preservar o resultado de uma multiplicação, cada posição da memória vetorial armazena uma palavra de 64 bits, composta por quatro *lanes* de 16 bits. Nas escritas do host, os quatro valores de 8 bits são estendidos com sinal antes de serem gravados. O conjunto inicial de operações inclui adição, subtração, multiplicação e divisão; `rsp_error` é ativado para divisão por zero ou *overflow* signed.

2 Linguagem utilizada

O projeto será desenvolvido em **SystemVerilog**. A linguagem será utilizada tanto para a descrição dos módulos de hardware quanto para a implementação do ambiente de verificação.

O ambiente de verificação UVM também será implementado em SystemVerilog.

Tabela 1: Uso de SystemVerilog no projeto.

Parte do projeto	**Utilização**
RTL	Descrição sintetizável dos módulos que formam a mini GPU.
Testbench Inicial	Geração dos pacotes de 42 bits e dos demais estímulos de entrada.
Assertions	Verificação dos handshakes, sinais de controle e transições da FSM.
Cobertura	Verificação dos opcodes, tipos de pacote, endereços e casos de borda exercitados durante os testes.

O código executado pelo host não faz parte do escopo inicial da implementação. Para a verificação, o comportamento do host será representado pelo testbench, que enviará diretamente os pacotes de requisição de 42 bits.

2.1 Parâmetros da arquitetura

Tabela 2: Parâmetros principais da mini GPU.

Parâmetro	Valor
Interface de requisição do host	Pacote de 42 bits com protocolo
Interface de resposta ao host	Dados de 64 bits e resposta de um comando
Barramento interno	AXI4-Lite simplificado, dados vetoriais de 64 bits
Número de *lanes*	4
Largura de cada elemento	Entrada do host: int8; *lane* interna e resultado: int16, signed
Palavra vetorial	64 bits
Memória vetorial	256 palavras de 64 bits
Endereço interno da memória	8 bits
Pacote interno de requisição	42 bits: `TYPE[1:0]` + `PAYLOAD[39:0]`
Número de comandos simultâneos	1

3 Arquitetura

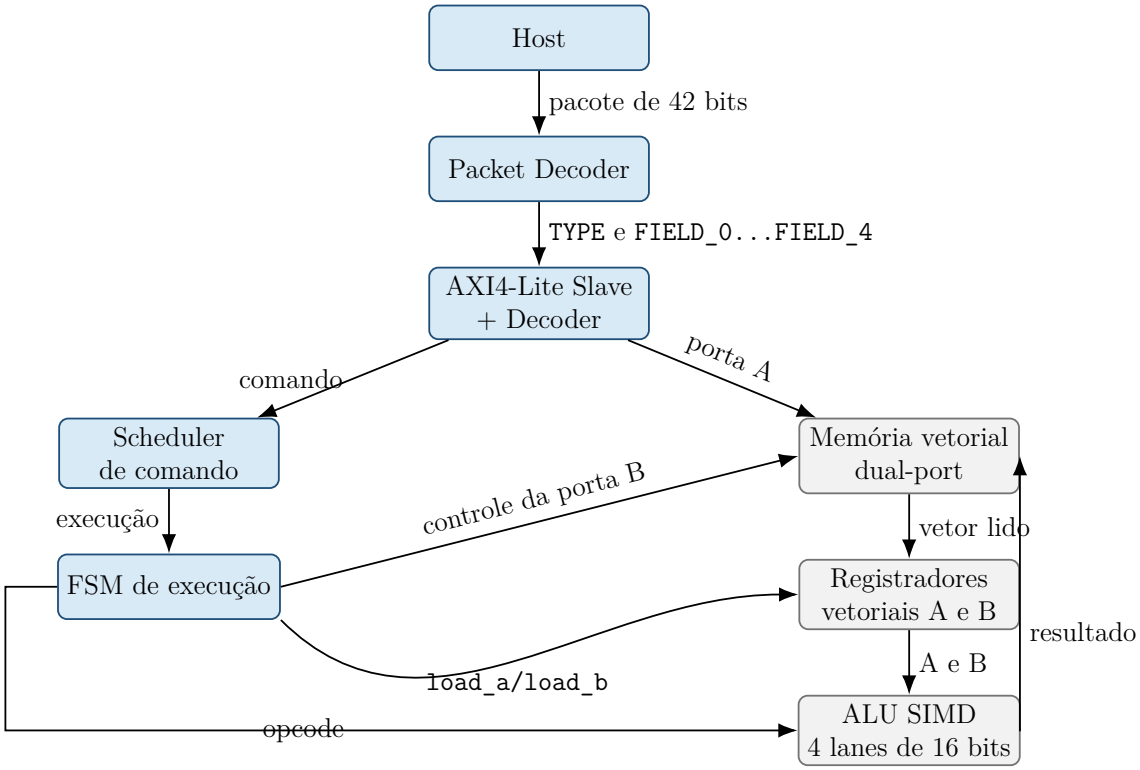


Figure 1: Arquitetura funcional da mini GPU SIMD.

3.1 Descrição dos blocos

3.1.1 Packet Decoder

É a porta de entrada das requisições do host. O bloco recebe o pacote de 42 bits. O Packet Decoder não acessa a memória nem o Scheduler diretamente; sua única responsabilidade é entregar uma representação decodificada e estável diretamente ao AXI4-Lite Slave Decoder.

3.1.2 AXI4-Lite Slave Decoder

Recebe diretamente do Packet Decoder o tipo e os cinco campos desempacotados. O bloco interpreta a requisição, calcula o endereço interno correspondente e seleciona `CONTROL` e `COMMAND` ou a memória vetorial.

3.1.3 Memória vetorial dual-port

Memória com duas interfaces independentes de leitura e escrita. A porta A é controlada pelo AXI4-Lite Slave Decoder, e a porta B é controlada pela FSM. Cada endereço armazena um vetor completo de 64 bits, formado por quatro *lanes* de 16 bits.

3.1.4 Scheduler de comando

Recebe do AXI4-Lite Slave Decoder o opcode, os endereços dos vetores A e B e o endereço de destino. O Scheduler armazena um comando, controla os sinais `BUSY` e `DONE`, via registrador de STATUS e entrega o comando à FSM. `valid/ready`.

3.1.5 FSM de execução

Controla a leitura dos dois operandos, o carregamento dos registradores, a execução da ALU e a escrita do resultado. A sequência de estados é apresentada na Tabela 3.

Tabela 3: Estados da FSM de execução.

Estado	**Descrição**
`IDLE`	Aguarda um comando válido do Scheduler.
`READ_A`	Coloca `SRC_A` no endereço de leitura da memoria porta B e solicita leitura.
`LOAD_A`	Ativa `load_a` e carrega no registrador o vetor devolvido pela memoria.
`READ_B`	Coloca `SRC_B` no endereço de leitura da memoria porta B e solicita leitura.
`LOAD_B`	Ativa `load_b` e carrega no registrador o vetor devolvido pela memoria.
`EXECUTE`	Mantém os operandos e o opcode estáveis para a ALU SIMD.
`WRITE_RESULT`	Escreve o resultado da ALU no endereço `DST`.
`FINISH`	Gera `exec_done` por um ciclo e retorna a `IDLE`.

3.1.6 Registradores vetoriais A e B

Armazenam os dois vetores lidos da porta B da memória. O barramento de 64 bits proveniente da memória alimenta os dois registradores, e a FSM seleciona o destino por meio de `load_a` e `load_b`.

3.1.7 ALU SIMD com quatro lanes de 16 bits

Composta por quatro ALUs simples de 16 bits. As quatro unidades recebem o mesmo opcode e operam simultaneamente sobre posições independentes dos vetores A e B. Os operandos enviados pelo host são originalmente de 8 bits e são estendidos com sinal na escrita da memória; assim, uma multiplicação de 8 bits preserva seu resultado de 16 bits.

4 Interfaces dos blocos

As tabelas apresentam somente os sinais necessários entre os blocos. Barramentos de dados são conectados diretamente ao bloco que os utiliza; a FSM permanece responsável apenas pelos sinais de controle.

4.1 Packet Decoder

Tabela 4: Interface do Packet Decoder.

Sinal	Tipo	Largura	Descrição
`clk/rst`	Input	1 cada	Clock e reset do bloco.
`req_payload`	Input	42	Pacote de requisição enviado pelo host.
`req_valid/ready`	Input/Output	1 cada	Handshake com o host.
`decoded_valid/ready`	Output/Input	1 cada	Handshake com o AXI4-Lite Slave Decoder.
`req_type`	Output	2	Cópia de `req_payload[41:40]`.
`field_0` a `field_4`	Output	5×8	Cópias dos cinco campos de 8 bits do pacote.

4.2 AXI4-Lite Slave Decoder

Tabela 5: Interface do AXI4-Lite Slave Decoder.

Sinal	Tipo	Largura	Descrição
`clk/rst`	Input	1 cada	Clock e reset.
`decoded_valid/ready`	Input/Output	1 cada	Handshake com o Packet Decoder.
`req_type + fields`	Input	2 e 5×8	Tipo e cinco campos desempacotados.
`rsp_*`	Output	64, 1 e 1	`rsp_data`, `rsp_valid` e `rsp_error`.
`mem_a_addr/wdata`	Output	8 e 64	Endereço e vetor de quatro *lanes* de 16 bits enviados à porta A.
`mem_a_we/re`	Output	1 cada	Habilitações da porta A.
`mem_a_rdata`	Input	64	Vetor lido pela porta A.
`cmd_fields`	Output	32	OPCODE, SRCA, SRCB, DST
`cmd_valid`	Output	1	Pulso enviado ao Scheduler quando chega `GPU_START`.
`cmd_ready/busy/done`	Input	1 cada	Prontidão e estado fornecidos pelo Scheduler.

4.3 Scheduler

Tabela 6: Interface do Scheduler.

Sinal	Tipo	Largura	Descrição
`clk/rst`	Input	1 cada	Clock e reset.
`cmd_valid/ready`	Input/Output	1 cada	Handshake de entrada do comando.
`cmd_fields`	Input	32	OPCODE, SRCA, SRCB, DST.
`exec_valid/ready`	Output/Input	1 cada	Handshake com a FSM.
`exec_fields`	Output	32	Comando armazenado entregue à FSM.
`exec_done`	Input	1	Pulso de término gerado pela FSM.
`busy/done`	Output	1 cada	Estado global da execução.

4.4 FSM de execução

Tabela 7: Interface da FSM de execução.

Sinal	Tipo	Largura	Descrição
`clk/rst`	Input	1 cada	Clock e reset.
`exec_valid/ready`	Input/Output	1 cada	Handshake com o Scheduler.
`exec_fields`	Input	32	OPCODE, SRCA, SRCB, DST recebido do Scheduler.
`mem_b_addr`	Output	8	Endereço da porta B da memória.
`mem_b_we/re`	Output	1 cada	Controles de escrita e leitura da porta B.
`load_a/load_b`	Output	1 cada	Habilitações dos registradores vetoriais.
`alu_opcode`	Output	8	Opcode aplicado à ALU SIMD.
`exec_done`	Output	1	Pulso de término da execução.

O barramento `mem_b_rdata` liga diretamente a memória aos registradores A e B. Da mesma forma, `result` da ALU liga-se diretamente a `b_wdata` da memória. Portanto, esses dados não passam pela FSM.

4.5 Memória vetorial dual-port

Tabela 8: Interface da memória vetorial dual-port.

Sinal	Tipo	Largura	Descrição
`clk`	Input	1	Clock da memória.
`a_addr/b_addr`	Input	8 cada	Endereços das portas A e B.
`a_wdata/b_wdata`	Input	64 cada	Dados de escrita das portas.
`a_we/b_we`	Input	1 cada	Habilitações de escrita.
`a_re/b_re`	Input	1 cada	Habilitações de leitura.
`a_rdata/b_rdata`	Output	64 cada	Dados lidos nas duas portas.

4.6 Registradores vetoriais A e B

Tabela 9: Interface dos registradores vetoriais.

Sinal	Tipo	Largura	Descrição
`clk/rst`	Input	1 cada	Clock e reset.
`mem_b_rdata`	Input	64	Vetor retornado diretamente pela porta B da memória.
`load_a/load_b`	Input	1 cada	Selecionam qual registrador captura `mem_b_rdata`.
`vector_a/vector_b`	Output	64 cada	Operandos enviados à ALU SIMD.

4.7 ALU SIMD

Tabela 10: Interface da ALU SIMD com quatro lanes.

Sinal	Tipo	Largura	Descrição
`vector_a/vector_b`	Input	64 cada	Dois vetores com quatro *lanes* de 16 bits.
`opcode`	Input	8	Operação comum às quatro lanes.
`result`	Output	64	Resultado formado por quatro *lanes* de 16 bits.

5 Mapa de endereços AXI

Tabela 11: Mapa de endereços do periférico.

Endereço	**Nome**	**Função**
0x000	CONTROL	Bit 0: solicitação de início da execução.
0x004	STATUS	Bit 0: BUSY; bit 1: DONE.
0x008	COMMAND	Opcode, SRC_A, SRC_B e DST.
0x100-0x4FC	Memória vetorial	256 palavras de 64 bits.

O mapa de endereços é interno à mini GPU. O host não produz endereços AXI; ele envia somente o pacote de 42 bits. O Packet Decoder desempacota os campos, e o AXI4-Lite Slave Decoder calcula o endereço e os dados da operação interna.

6 Estrutura de comandos

O host não acessa diretamente os canais AXI4-Lite. Toda requisição entra na mini GPU pelo Packet Decoder no formato único de 42 bits descrito a seguir. Os campos extraídos são encaminhados diretamente ao AXI4-Lite Slave Decoder, responsável pela tradução para o mapa interno de endereços.

6.1 Formato geral do pacote

Tabela 12: Formato geral do pacote interno de 42 bits.

TYPE	FIELD_0	FIELD_1	FIELD_2	FIELD_3	FIELD_4
[41:40]	[39:32]	[31:24]	[23:16]	[15:8]	[7:0]

Tabela 13: Tipos de requisição aceitos.

Tipo	Nome	Operação
2'b00	MEM_READ	Ler uma posição da memória vetorial.
2'b01	MEM_WRITE	Escrever um vetor na memória.
2'b10	GPU_COMMAND	Configurar a operação SIMD.
2'b11	GPU_START	Iniciar a operação configurada.

6.2 Comando de operação da GPU

Tabela 14: Formato de GPU_COMMAND.

GPU_COMMAND	Reservado	OPCODE	SRC_A	SRC_B	DST
[41:40]	[39:32]	[31:24]	[23:16]	[15:8]	[7:0]

6.3 Escrita de vetor

Tabela 15: Formato de MEM_WRITE.

MEM_WRITE	MEM_ADDR	LANE_3	LANE_2	LANE_1	LANE_0
[41:40]	[39:32]	[31:24]	[23:16]	[15:8]	[7:0]

Os campos LANE_0 a LANE_3 do pacote continuam tendo 8 bits, pois representam os valores de entrada do host. No MEM_WRITE, o AXI4-Lite Slave Decoder estende cada campo com sinal e grava as quatro *lanes* de 16 bits na palavra vetorial de 64 bits.

6.4 Leitura de vetor

Tabela 16: Formato da requisição `MEM_READ`.

`MEM_READ`	`MEM_ADDR`	Reservado
`[41:40]`	`[39:32]`	`[31:0]`

A resposta de leitura é retornada em `rsp_data[63:0]`, acompanhada por `rsp_valid`. Caso o AXI retorne uma resposta de erro, o adaptador também ativa `rsp_error`.

6.5 Início da operação

Tabela 17: Formato de `GPU_START`.

`GPU_START`	Reservado
`[41:40]`	`[39:0]`

O campo reservado deve ser escrito como zero. O próprio tipo `GPU_START` representa a solicitação de início. Para simplificar a interface do host, `rsp_valid` é gerado para esse pacote somente quando a FSM concluir a operação e o Scheduler produzir `DONE`.

7 ISA

Tabela 18: Conjunto inicial de instruções.

Comando	**Opcode binário**	**Especificação**
`ADD`	`0000 0001`	Soma os elementos correspondentes dos dois vetores.
`SUB`	`0000 0010`	Subtrai os elementos correspondentes dos dois vetores.
`MUL`	`0000 0011`	Multiplica os elementos correspondentes. Para operandos de entrada de 8 bits, preserva o produto de 16 bits.
`DIV`	`0000 0100`	Realiza divisão inteira.

8 Lista de funcionalidades

8.1 Modulo completo

[GPU_01_RESET]
A mini GPU deve retornar ao estado ocioso após o reset. Os sinais de controle, `BUSY` e `DONE` devem ser desativados.

[GPU_02_END_TO_END]
A mini GPU deve receber pelo host pacotes de 42 bits para escrever dois vetores, configurar um comando, iniciar a execução, armazenar o resultado no endereço de destino e devolvê-lo em uma requisição de leitura.

8.2 Packet Decoder

[PKT_01_HANDSHAKE]
O Packet Decoder deve capturar `req_payload` somente quando ocorrer `req_valid e req_ready`.

[PKT_02_FIELDS]
O bloco deve copiar `req_payload[41:40]` para `TYPE` e extrair os cinco campos de 8 bits conforme a Tabela 12.

[PKT_03_OUTPUT]
Após aceitar uma requisição, o bloco deve ativar `decoded_valid` e manter todos os campos estáveis até ocorrer `decoded_valid e decoded_ready`.

[PKT_04_UNDERCOMMAND]
Enquanto houver uma requisição decodificada ainda não aceita pelo AXI4-Lite Slave Decoder, o Packet Decoder deve manter desativado `req_ready`.

[PKT_05_HOST_STABLE]
Quando `req_valid=1` e `req_ready=0`, o host deve manter `req_payload` e `req_valid` estáveis até a requisição ser aceita.

8.3 AXI4-Lite Slave Decoder

[AXI_01_DECODE]
O bloco deve interpretar `req_type` e os cinco campos recebidos diretamente do Packet Decoder, direcionando a requisição para `CONTROL`, `STATUS`, `COMMAND` ou para a porta A da memória vetorial.

[AXI_02_MEM_WRITE]
Para `MEM_WRITE`, o bloco deve usar `FIELD_0` como endereço da memória e formar uma palavra vetorial de 64 bits, estendendo com sinal os quatro campos de entrada de 8 bits `FIELD_1` a `FIELD_4`.

[AXI_03_MEM_READ]
Para `MEM_READ`, o bloco deve ler pela porta A o endereço indicado por `FIELD_0` e retornar a palavra em `rsp_data`.

`[AXI_04_COMMAND]`
Para `GPU_COMMAND`, o bloco deve armazenar corretamente `OPCODE`, `SRC_A`, `SRC_B` e `DST`.

`[AXI_05_CONTROL_STATUS]`
Para `GPU_START`, o bloco deve solicitar a execução e acompanhar os sinais `BUSY` e `DONE` do Scheduler.

`[AXI_06_SINGLE_REQUEST]`
O bloco deve manter no máximo uma requisição ou execução da GPU pendente.

`[AXI_07_RESPONSE]`
O bloco deve montar o pacote de resposta ao host com o resultado da ALU.

8.4 Scheduler

`[SCH_01_HANDSHAKE]`
O Scheduler deve aceitar um comando somente quando ocorrer `cmd_valid e cmd_ready`.

`[SCH_02_DISPATCH]`
O Scheduler deve armazenar o comando aceito e entregá-lo integralmente à FSM usando `exec_valid` e `exec_ready`.

`[SCH_03_STATUS]`
O Scheduler deve ativar `BUSY` após aceitar um comando, desativá-lo após `exec_done` e ativar `DONE` quando a operação terminar.

8.5 Memória vetorial dual-port

`[MEM_01_PORT_A]`
Todos os 256 endereços devem ser legíveis e graváveis pela porta A, controlada pelo AXI4-Lite Slave Decoder.

`[MEM_02_PORT_B]`
Todos os 256 endereços devem ser legíveis e graváveis pela porta B, controlada pela FSM.

`[MEM_03_DUAL_PORT]`
As duas portas devem operar independentemente quando acessarem endereços diferentes.

`[MEM_04_SYNC_READ]`
Uma leitura deve disponibilizar o dado de acordo com a latência síncrona definida para a memória.

8.6 FSM de execução

`[FSM_01_ACCEPT]`
A FSM deve aceitar um novo comando somente no estado `IDLE`.

[FSM_02_READ_A]
A FSM deve solicitar a leitura do endereço `SRC_A` e posteriormente carregar o registrador A.

[FSM_03_READ_B]
A FSM deve solicitar a leitura do endereço `SRC_B` e posteriormente carregar o registrador B.

[FSM_04_EXECUTE]
A FSM deve enviar o opcode correto à ALU depois que os dois operandos estiverem carregados.

[FSM_05_WRITEBACK]
A FSM deve escrever o resultado da ALU no endereço `DST`.

[FSM_06_SEQUENCE]
A FSM deve percorrer somente a sequência válida: `IDLE`, `ISSUE_READ_A`, `CAPTURE_A`, `ISSUE_READ_B`, `CAPTURE_B`, `EXECUTE`, `WRITE_RESULT`, `FINISH` e `IDLE`.

[FSM_07_DONE]
A FSM deve gerar `exec_done` durante um único ciclo após escrever o resultado.

8.7 Registradores vetoriais A e B

[REG_01_LOAD_A]
Quando `load_a=1`, o registrador A deve capturar `mem_rdata`, sem alterar o registrador B.

[REG_02_LOAD_B]
Quando `load_b=1`, o registrador B deve capturar `mem_rdata`, sem alterar o registrador A.

[REG_03_HOLD]
Quando `load_a=0` e `load_b=0`, os dois registradores devem manter seus valores.

8.8 ALU SIMD com quatro lanes de 16 bits

[ALU_01_SOMA]
A ALU SIMD deve executar a adição elemento a elemento entre dois vetores.

[ALU_02_SUB]
A ALU SIMD deve executar a subtração elemento a elemento entre dois vetores.

[ALU_03_MULT]
A ALU SIMD deve executar a multiplicação elemento a elemento e preservar, em cada *lane* de 16 bits, o produto de dois operandos de entrada de 8 bits.

[ALU_04_DIV]
A ALU SIMD deve executar a divisão inteira signed elemento a elemento, produzindo zero e sinalizando `rsp_error` quando o divisor de uma lane for zero.

`[ALU_06_ERROR]`
A ALU SIMD deve sinalizar `rsp_error` na resposta de `GPU_START` quando houver divisão por zero ou *overflow* signed em qualquer lane.

`[ALU_05_SIMD]`
As quatro lanes devem receber o mesmo opcode e produzir resultados independentes, sem dependência entre as posições do vetor.

9 Lista de testes

Tabela 19: Lista consolidada de testes e rastreabilidade com as funcionalidades.

Teste	Descrição	Funcionalidades cobertas	Prioridade
`test_gpu_reset`	Aplica reset durante o estado ocioso e durante uma operação, verificando o retorno ao estado inicial e a desativação dos sinais de controle.	[GPU_01_RESET]	Alta
`test_packet_field_extraction`	Aplica pacotes com padrões distintos nos seis campos e verifica a extração exata de `TYPE` e de `FIELD_0` a `FIELD_4`.	[PKT_01_HANDSHAKE], [PKT_02_FIELDS]	Alta
`test_host_backpressure`	Apresenta um segundo pacote `MEM_WRITE` enquanto o Packet Decoder está ocupado e verifica que o host mantém `req_valid` e `req_payload` estáveis até `req_ready` voltar a 1.	[PKT_01_HANDSHAKE], [PKT_04_UNDERCOMMAND], [PKT_05_HOST_STABLE]	Alta
`test_packet_mem_write`	Envia um pacote `MEM_WRITE` e verifica `mem_a_addr`, `mem_a_wdata`, `mem_a_we` e a confirmação ao host.	[AXI_02_MEM_WRITE], [AXI_06_SINGLE_REQUEST], [AXI_07_RESPONSE]	Alta
`test_packet_mem_read`	Envia um pacote `MEM_READ`, fornece um valor conhecido em `mem_a_rdata` e verifica `rsp_data`, `rsp_valid` e a propagação de erro.	[AXI_03_MEM_READ], [AXI_07_RESPONSE]	Alta
`test_packet_command_start`	Envia `GPU_COMMAND` seguido de `GPU_START`; verifica os campos entregues ao Scheduler, o pulso `cmd_valid` e a resposta somente após `DONE`.	[AXI_04_COMMAND], [AXI_05_CONTROL_STATUS], [AXI_06_SINGLE_REQUEST], [AXI_07_RESPONSE]	Alta
`test_axi_decode`	Aplica os quatro valores de `req_type` e verifica se cada requisição seleciona corretamente a memória, o registrador de comando ou o registrador de controle.	[AXI_01_DECODE], [AXI_04_COMMAND], [AXI_05_CONTROL_STATUS]	Alta
`test_axi_memory_access`	Escreve e lê vetores pelos endereços inicial, intermediário e final da memória.	[AXI_02_MEM_WRITE], [AXI_03_MEM_READ], [MEM_01_PORT_A]	Alta
`test_scheduler_handshake`	Verifica o protocolo `valid/ready`, o armazenamento do comando, os sinais `BUSY` e `DONE` e a rejeição de um novo comando enquanto o Scheduler estiver ocupado.	[SCH_01_HANDSHAKE] a [SCH_04_SINGLE_COMMAND]	Alta

Continua na próxima página

Continuação da Tabela 19

Teste	Descrição	Funcionalidades cobertas	Prioridade
`test_fsm_sequence`	Verifica todos os estados e transições, os endereços apresentados à memória e os pulsos `load_a`, `load_b` e `exec_done`.	`[FSM_01_ACCEPT]` a `[FSM_07_DONE]`	Alta
`test_gpu_add_end_to_end`	Envia dois pacotes `MEM_WRITE`, um `GPU_COMMAND` com `ADD`, um `GPU_START` e, após o término, um `MEM_READ` para obter o resultado.	`[GPU_02_END_TO_END]` e integração dos blocos	Alta
`test_gpu_all_ops`	Executa pelo protocolo de pacotes uma operação completa para cada opcode suportado. Os operandos são tratados como valores *signed*; a divisão por zero deve retornar resultado zero, `DONE` e `rsp_error=1`.	`[GPU_02_END_TO_END]`, `[ALU_01_SOMA]` a `[ALU_04_DIV]`, `[ALU_06_ERROR]`	Alta
`test_gpu_mul_8bit`	Escreve o valor mínimo signed de 8 bits (`8'h80`, ou -128) nas quatro *lanes* de dois vetores, executa `MUL` e verifica o produto `16'h4000` em cada *lane* da resposta de 64 bits.	`[GPU_02_END_TO_END]`, `[AXI_02_MEM_WRITE]`, `[AXI_03_MEM_READ]`, `[ALU_03_MULT]`, `[ALU_05_SIMD]`	Alta
`gpu_matrix_mul_test`	Executa a multiplicação de duas matrizes 4x4 por meio de operações SIMD `MUL` e `ADD`, gravando e lendo a matriz resultante pela memória vetorial.	`[GPU_02_END_TO_END]`, `[ALU_01_SOMA]`, `[ALU_03_MULT]`, `[ALU_05_SIMD]`	Alta
`test_gpu_back_to_back`	Executa várias sequências de pacotes, verificando que uma nova requisição só é aceita após a conclusão da anterior e que os resultados não são sobrescritos.	`[PKT_04_UNDERCOMMAND]`, `[AXI_06_SINGLE_REQUEST]`, `[SCH_03_STATUS]`, `[GPU_02_END_TO_END]`	Média
`test_gpu_address_corner_cases`	Utiliza os endereços 0 e 255, `SRC_A=SRC_B` e `DST` igual a um dos endereços dos operandos.	`[MEM_01_PORT_A]`, `[MEM_02_PORT_B]`, `[FSM_02_READ_A]`, `[FSM_03_READ_B]`, `[FSM_05_WRITEBACK]`	Média
`test_gpu_random`	Gera vetores, operações e endereços aleatórios e compara o resultado final com um modelo de referência.	Todas as funcionalidades válidas	Alta

10 Critérios iniciais de conclusão

- Todos os testes de prioridade alta devem ser aprovados.
- Todas as funcionalidades listadas devem estar associadas a pelo menos um teste.
- A cobertura funcional deve contemplar todos os opcodes, endereços de borda, valores zero, valores signed de interesse e `rsp_error`.
- A cobertura de código deve ser analisada para linhas, blocos, expressões, alternâncias e estados da FSM.

11 Próxima versão

- Revisão das interfaces de cada bloco, verificação de necessidade de cada entrada e saida.
- Implementação do módulo RTL para teste de conceito
- Implementação do testbench para o módulo

12 Histórico de versões

Versão	Modificação
1.1	Testes de módulos internos removidos e teste GPU MUL adicionado ao plano de verificação UVM.
1.2	Caminho vetorial ampliado de 32 para 64 bits, com quatro *lanes* internas de 16 bits e entradas do host mantidas em 8 bits. Adicionado o teste `test_gpu_mul_8bit`, que valida `8'hFF * 8'hFF = 16'hFE01` em todas as *lanes*.
1.3	Operandos do host definidos como int8 em complemento de dois, com extensão de sinal para as lanes int16. Adicionado `rsp_error` para divisão por zero e overflow signed.

Tabela 20: Histórico de versões do plano de verificação.